

Expense Tracker Application

Full-Stack Implementation Plan

Generated: August 03, 2026 at 05:54 PM

A production-ready, cloud-deployable expense tracker with multi-user collaboration, role-based access control, recurring transaction detection, advanced budget management, and comprehensive analytics.

Expense Tracker - Implementation Plan

Problem Statement

Build a production-ready, cloud-deployable expense tracker application that supports multi-user collaboration with role-based access control. The system must handle complex financial workflows including recurring transaction detection, advanced budget management with rollover capabilities, predictive spending insights, and comprehensive data visualization with export functionality.

The application should enable users to:

- Track personal and shared household expenses
- Collaborate with family members or groups with granular permissions
- Automatically detect and manage recurring transactions
- Set and monitor budgets with rollover capabilities
- Gain insights through analytics and predictions
- Export financial data in multiple formats

Requirements

Functional Requirements

1. Authentication & Authorization

- JWT-based stateless authentication with access tokens
- Role-based access control (Owner/Admin/Viewer) for shared resources
- User registration and login with secure password hashing (BCrypt)
- Token validation and refresh mechanisms

2. Multi-User Sharing

- Users can create and manage household/group budgets
- Invite members with specific roles (Owner, Admin, Viewer)
- Granular permissions for viewing, editing, and deleting transactions
- Household member management and role updates

3. Transaction Management

- Full CRUD operations on expenses and income
- Category management with customizable icons and colors
- Advanced filtering (date ranges, categories, amount, transaction type)
- Recurring transaction detection and automation
- Tag system for flexible organization
- Search functionality by description and merchant name

4. Budget Features

- Monthly category-based budget limits
- Budget rollover functionality for unused amounts
- Budget templates for reuse across months
- Overspending alerts and notifications (80% warning, 100% exceeded)
- Shared budgets across household members
- Historical budget tracking and analysis

Expense Tracker - Implementation Plan

5. Analytics & Insights

- Dashboard with summary cards (balance, monthly income/expense, top categories)
- Category breakdown visualizations with pie charts
- Monthly trend analysis with line and bar charts
- Custom date range comparisons
- Rule-based predictive spending insights using pattern detection
- Recurring expense identification and forecasting
- Anomaly detection for unusual spending patterns

6. Export & Reporting

- CSV export for transactions with customizable filters
- PDF report generation with charts and summaries
- Customizable date range exports
- Budget reports showing spent vs. allocated amounts

Technical Requirements

- Backend: Spring Boot 3.x, Java 17+, PostgreSQL 15+
- Frontend: React 18+, TypeScript 5.x, Vite, Tailwind CSS, shadcn/ui
- Testing: JUnit 5 + Mockito + Testcontainers (backend), Vitest + Testing Library (frontend)
- Database Migrations: Flyway for versioned schema management
- Deployment: Dockerized services with separate deployments
- Frontend Hosting: Vercel or Netlify
- Backend Hosting: Railway or Render
- Database: Managed PostgreSQL (Neon, Supabase, or Railway)
- Development: Docker Compose for local orchestration

Background & Research Findings

TanStack Query Best Practices

- Use hierarchical query keys for cache organization: `['transactions', userId, { filters }]`
- Implement centralized error handling with QueryClient defaults
- Minimize `useEffect/useState` for server state management
- Implement optimistic updates for better user experience
- Proper TypeScript integration with typed query and mutation hooks
- Use `staleTime` and `cacheTime` strategically for performance

Spring Boot JWT Security

- Custom JWT filter extending `OncePerRequestFilter`
- Stateless session management with Spring Security
- JJWT library (0.12.x) for token generation and validation
- BCrypt password encoding with appropriate strength (12)
- `SecurityFilterChain` with role-based endpoint protection
- Proper exception handling for authentication failures

Flyway Migration Strategy

- Versioned migrations with naming convention: `V1__description.sql`
- Separate migration user with elevated database privileges
- Application user with restricted CRUD permissions only
- Repeatable migrations (`R__`) for views and procedures
- Baseline existing databases before applying migrations
- Use checksums to detect manual schema changes

Docker Compose Development

- Spring Boot's `spring-boot-docker-compose` dependency for auto-connection
- PostgreSQL with persistent volumes for data retention
- Health checks using `pg_isready` for proper startup sequencing
- `depends_on` with condition: `service_healthy` for reliability
- Environment-specific profiles (`dev`, `test`, `prod`)
- Separate networks for service isolation

Expense Tracker - Implementation Plan

Recurring Transaction Detection

- Rule-based pattern matching for merchant names and amounts
- Configurable tolerance thresholds ($\pm 10\%$ amount variance, ± 3 days interval)
- Rolling window analysis examining 3-6 month transaction history
- Confidence scoring based on pattern consistency
- Support for weekly, monthly, quarterly, and annual patterns
- Fuzzy string matching (Levenshtein distance) for merchant name matching

shadcn/ui Integration

- Copy-paste components directly into project (no external dependencies)
- Class Variance Authority (CVA) for flexible variant management
- Built on Radix UI primitives for accessibility compliance
- Tailwind CSS design tokens (primary, secondary, border, etc.)
- Compound component patterns for complex UI structures
- Easy customization through direct source code access

Role-Based Access Control

- Spring Security `@PreAuthorize` expressions for method-level security
- Custom `PermissionEvaluator` for household-specific access checks
- Three-tier role system: Owner (full control), Admin (manage content), Viewer (read-only)
- Aspect-oriented approach for automatic household context injection
- Global exception handling for access denied scenarios
- Database-driven permissions for flexibility

Proposed Solution Architecture

System Overview

The application follows a modern three-tier architecture with clear separation of concerns:

Frontend Layer: React-based SPA hosted on Vercel/Netlify, communicating with the backend via RESTful APIs. Uses TanStack Query for efficient data fetching and caching.

Backend Layer: Spring Boot REST API hosted on Railway/Render, handling business logic, authentication, and data persistence. Implements JWT-based stateless authentication.

Data Layer: Managed PostgreSQL database with Flyway-managed schema migrations, ensuring data consistency and versioning.

Key Architectural Decisions

1. **Stateless Authentication:** JWT tokens eliminate server-side session storage, enabling horizontal scaling
2. **Separate Repositories:** Frontend and backend maintained independently for clear ownership and deployment flexibility
3. **Role-Based Multi-Tenancy:** Household model enables resource sharing while maintaining security boundaries
4. **Rule-Based Analytics:** Simple algorithms for predictions avoid ML complexity while providing valuable insights
5. **Event-Driven Budget Alerts:** Scheduled jobs check budget thresholds and trigger notifications asynchronously
6. **Optimistic UI Updates:** Frontend updates immediately while backend processes requests in background

Expense Tracker - Implementation Plan

Database Schema Design

The database schema is designed to support multi-user collaboration with clear ownership and permission boundaries:

Core Entities:

- Users: Authentication credentials and profile information
- Households: Shared workspaces for family/group expense tracking
- UserHouseholds: Join table with role assignments (Owner/Admin/Viewer)
- Categories: Expense/income categories with customization options
- Transactions: Financial records linked to users, households, and categories
- RecurringPatterns: Detected recurring transaction patterns
- Budgets: Monthly spending limits per category
- BudgetAlerts: Triggered notifications for budget thresholds

Key Relationships:

- One-to-Many: User -> Transactions, Household -> Budgets
- Many-to-Many: User <-> Household (through UserHouseholds with roles)
- One-to-Many: Category -> Transactions, Category -> Budgets
- One-to-Many: RecurringPattern -> Transactions

Security Architecture

Authentication Flow:

1. User submits credentials to /api/auth/login
2. Backend validates credentials against BCrypt hash
3. JWT token generated with user ID and roles as claims
4. Frontend stores token in localStorage
5. All subsequent requests include token in Authorization header
6. Backend validates token and extracts user context

Authorization Flow:

1. User attempts to access household resource
2. Backend extracts user ID from JWT token
3. Permission service checks user's role in target household
4. Access granted/denied based on role and operation type
5. Owner: Full access, Admin: Manage content, Viewer: Read-only

Technology Stack

Frontend Technologies

- Build Tool: Vite 5.x - Fast development server with HMR
- Framework: React 18+ - Component-based UI library
- Language: TypeScript 5.x - Static typing for JavaScript
- Styling: Tailwind CSS 3.x - Utility-first CSS framework
- UI Components: shadcn/ui - Accessible components built on Radix UI
- State Management: TanStack Query v5 - Server state management
- HTTP Client: Axios - Promise-based HTTP client with interceptors
- Routing: React Router v6 - Declarative routing for React
- Forms: React Hook Form + Zod - Type-safe form validation
- Charts: Recharts - Composable charting library
- Icons: Lucide React - Consistent icon set
- Utilities: clsx, tailwind-merge - CSS class manipulation
- Notifications: Sonner - Toast notifications
- Testing: Vitest, Testing Library, MSW - Unit and integration testing

Backend Technologies

- Framework: Spring Boot 3.2+ - Production-ready framework
- Language: Java 17+ - LTS version with modern features
- Security: Spring Security 6.x + JWT 0.12.x - Authentication and authorization
- Data Access: Spring Data JPA + Hibernate 6.x - ORM and data persistence
- Database: PostgreSQL 15+ - Relational database with JSON support
- Migrations: Flyway 10.x - Database version control
- Validation: Jakarta Validation - Bean validation annotations
- Documentation: Springdoc OpenAPI 2.x - Swagger UI generation
- Utilities: Lombok - Boilerplate code reduction
- Mapping: MapStruct - Type-safe bean mapping
- Testing: JUnit 5, Mockito, Testcontainers, REST Assured

DevOps & Deployment

- Containerization: Docker - Application containerization
- Local Development: Docker Compose - Multi-container orchestration
- CI/CD: GitHub Actions - Automated testing and deployment
- Frontend Hosting: Vercel or Netlify - Static site hosting with CDN
- Backend Hosting: Railway or Render - Container-based hosting
- Database: Neon, Supabase, or Railway - Managed PostgreSQL
- Version Control: Git - Source code management

Implementation Task Breakdown

The implementation is divided into 30 sequential tasks, organized into logical phases. Each task includes clear objectives, implementation steps, testing strategies, and deliverable demos.

Task 1: Project Initialization and Setup

Objective:

Set up foundational structure for frontend and backend with proper tooling

Implementation Steps:

- Initialize Spring Boot 3.2+ with dependencies: Web, Security, JPA, PostgreSQL, Flyway, Lombok, Validation, Springdoc
- Initialize Vite + React + TypeScript with strict configuration
- Configure Tailwind CSS and install shadcn/ui CLI
- Set up ESLint, Prettier for frontend; Checkstyle for backend
- Create .env.example files with required environment variables
- Initialize Git repositories (separate for frontend and backend)
- Configure basic CI workflows for linting and build checks

Testing:

Verify builds succeed, linting passes, development servers start without errors

Task 2: Database Schema Design and Migration Setup

Objective:

Design complete database schema with Flyway migrations

Implementation Steps:

- Create Flyway migration directory structure
- V1__create_users_table.sql: id, email, password_hash, first_name, last_name, timestamps
- V2__create_households_and_user_households.sql: households, user_households with role enum
- V3__create_categories_table.sql: with icons, colors, type enum
- V4__create_transactions_table.sql: with tags array, foreign keys
- V5__create_budgets_and_alerts.sql: budgets with rollover support
- V6__create_recurring_patterns_table.sql: pattern detection metadata
- Add indexes: user_id, household_id, transaction_date, category_id
- R__seed_default_categories.sql: repeatable migration for default data

Testing:

Run migrations against local PostgreSQL, verify schema correctness, check foreign key constraints

Expense Tracker - Implementation Plan

Task 3: Docker Compose Local Development Environment

Objective:

Create complete Docker Compose setup for local development

Implementation Steps:

- Create docker-compose.yml with PostgreSQL service (persistent volume, health check)
- Configure environment variables for database credentials
- Add spring-boot-docker-compose dependency to backend
- Create application-dev.yml for Docker connection
- Create application-local.yml for local PostgreSQL
- Add pgAdmin service for database management
- Create startup scripts for migrations and seed data
- Document connection strings in README

Testing:

Start Docker Compose, verify PostgreSQL accessible, Spring Boot connects, migrations run

Task 4: Backend - User Entity and Repository Layer

Objective:

Create JPA entities and repository interfaces for core domain model

Key Entities:

- User: Authentication and profile data
- Household: Shared workspace for expense tracking
- UserHousehold: Join table with role enum (OWNER, ADMIN, VIEWER)
- Category: Expense/income categories with icons and colors
- Transaction: Financial records with tags and relationships
- RecurringPattern: Detected recurring transaction metadata
- Budget & BudgetAlert: Budget limits and notifications

Task 5: Backend - JWT Authentication Service

Objective:

Implement JWT-based authentication with registration and login

Components:

- JwtService: Token generation and validation using JJWT library
- AuthenticationService: Registration and login business logic
- UserDetailsService: Load users from database for authentication
- JwtAuthenticationFilter: Validate tokens on each request
- SecurityFilterChain: Configure public and protected endpoints
- PasswordEncoder: BCrypt with strength 12
- DTOs: RegisterRequest, LoginRequest, AuthenticationResponse
- AuthenticationController: /api/auth/register and /api/auth/login endpoints

Task 6: Backend - Role-Based Access Control (RBAC)

Objective:

Implement granular permission checking for household-shared resources

Implementation:

- @PreAuthorize expressions for declarative security
- Custom PermissionEvaluator for household-specific checks
- HouseholdPermissionService: canView(), canEdit(), canDelete(), canManageMembers()
- Aspect for automatic household context injection from JWT
- @CurrentUser annotation for injecting authenticated user
- ResourceAccessDeniedException with global exception handler
- Method security configuration enabled

Task 7: Backend - Household Management API

Endpoints:

- POST /api/households - Create household (user becomes OWNER)
- GET /api/households - List user's households
- GET /api/households/{id} - Get household details
- POST /api/households/{id}/members - Invite member (OWNER/ADMIN)
- PATCH /api/households/{id}/members/{userId} - Update role (OWNER only)
- DELETE /api/households/{id}/members/{userId} - Remove member (OWNER/ADMIN)

Task 8: Backend - Category Management API

Features:

- CRUD operations for personal and household categories
- Icon picker integration with predefined icon set
- Color palette with customizable color selection
- Default system categories with special handling
- Category usage tracking (transaction count)
- Validation preventing deletion of categories with transactions

Task 9: Backend - Transaction Management API

Advanced Features:

- Dynamic filtering using JPA Criteria API or QueryDSL
- Filters: date range, category, type, tags, amount range, description search
- Pagination and sorting support (by date, amount, category)
- Tag management with PostgreSQL array field
- Bulk operations for efficient data management
- Transaction validation rules (amount > 0, valid date, etc.)

Task 10: Backend - Recurring Transaction Detection

Algorithm Steps:

- Group transactions by similar merchant names (fuzzy matching)
- Identify recurring intervals: weekly (7 ± 3 days), monthly (30 ± 5 days), quarterly (90 ± 7 days)
- Calculate average amounts with $\pm 10\%$ tolerance
- Compute confidence scores based on pattern consistency (0-1 scale)
- Store detected patterns with next expected date
- Scheduled job runs daily to detect new patterns
- Auto-create transactions for confirmed patterns
- User can confirm, dismiss, or disable auto-creation

Task 11: Backend - Budget Management with Rollover

Key Features:

- Monthly category-based budget limits
- Rollover calculation: unused amount carries to next month
- Alert thresholds: WARNING (80%), EXCEEDED (100%)
- Budget templates: save and apply configurations
- Scheduled job for month-end rollover processing
- Budget vs. actual spending comparison
- Historical budget tracking and trends

Task 12: Backend - Analytics and Insights Service

Analytics Endpoints:

- GET /api/analytics/dashboard - Summary metrics
- GET /api/analytics/category-breakdown - Category spending percentages
- GET /api/analytics/monthly-trends - 12-month income vs. expense
- GET /api/analytics/predictions - Rule-based spending forecasts

Prediction Algorithms:

- Simple moving average: 3-month average for category forecast
- Trend detection: identify increasing/decreasing/stable patterns
- Anomaly detection: flag transactions > 2 standard deviations from mean
- Seasonal patterns: identify recurring seasonal spending changes

Task 13: Backend - Export Service (CSV and PDF)

Export Functionality:

- CSV export: Transactions with all filters applied
- PDF report: Transaction list with summary statistics
- PDF budget report: Monthly budget vs. actual with charts
- Streaming response for large datasets
- Custom date range selection
- Formatted currency and date display
- Chart images embedded in PDF using chart libraries

Task 14: Backend - API Documentation with Swagger

Documentation Features:

- Springdoc OpenAPI 2.x integration
- Bearer token authentication in Swagger UI
- @Operation and @ApiResponse annotations on endpoints
- @Schema annotations on DTOs with examples
- API grouping by tags: Auth, Households, Transactions, etc.
- Interactive testing directly from Swagger UI
- OpenAPI 3.0 specification generation

Task 15: Backend - Integration Tests and Coverage

Testing Strategy:

- Testcontainers for PostgreSQL in test environment
- @SpringBootTest with MockMvc for controller tests
- Test authentication flows: register, login, token validation
- Test RBAC scenarios: different roles accessing resources
- Test transaction filtering with complex queries
- Test recurring pattern detection with sample data
- Test budget rollover and alert triggering
- REST Assured for API integration testing
- JaCoCo for code coverage reports (target: >80%)

Task 16: Frontend - Project Setup and UI Foundation

Setup Steps:

- Initialize Vite + React + TypeScript project
- Configure Tailwind CSS with custom design tokens
- Install shadcn/ui base components: Button, Card, Input, Dialog, etc.
- Set up React Router with protected and public routes
- Create ProtectedRoute wrapper for authentication
- Build layout components: AppLayout, Sidebar, Header
- Configure path aliases in tsconfig and vite.config

Task 17: Frontend - API Client and TanStack Query

Configuration:

- Create Axios instance with base URL from environment
- Request interceptor: attach JWT token from localStorage
- Response interceptor: handle 401 errors (redirect to login)
- QueryClient with default options (staleTime, retry)
- Hierarchical query keys for cache organization
- API service layer with typed methods
- Centralized error handling with Error Boundary
- AuthContext for user state and token management

Task 18: Frontend - Authentication UI

Components:

- LoginPage: Email/password form with validation
- RegisterPage: Name, email, password, confirm password
- Zod schemas for client-side validation
- React Hook Form integration for form state
- useLogin and useRegister mutation hooks
- Error handling with user-friendly messages
- Loading states with disabled form controls
- Password visibility toggle
- Redirect to dashboard after successful authentication

Expense Tracker - Implementation Plan

Task 19: Frontend - Dashboard Page

Dashboard Features:

- Summary cards: Total Balance, Monthly Income, Monthly Expense, Top Category
- useDashboardSummary query hook with caching
- Loading skeletons with shadcn/ui Skeleton
- Error state with retry button
- Household selector dropdown for multi-household users
- Date range picker (default: current month)
- Percentage change indicators with trend arrows
- Responsive grid layout

Task 20: Frontend - Transaction List with Filters

Features:

- useTransactions query hook with filter parameters
- Filter panel: date range, category, type, amount range, search
- shadcn/ui Table component for transaction display
- Pagination controls with page size selector
- Column sorting (click headers to sort)
- Row actions: Edit, Delete with confirmation dialog
- TransactionDialog for create/edit with validation
- Optimistic updates using TanStack Query mutations
- Empty state when no transactions found

Task 21: Frontend - Category Management

Task 22: Frontend - Budget Management

Task 23: Frontend - Analytics and Charts

Task 24: Frontend - Household Management

Tasks 21-24 follow similar patterns: query hooks for data fetching, shadcn/ui components for UI, React Hook Form for forms, and TanStack Query for state management. Each includes comprehensive error handling, loading states, and optimistic updates.

Task 25: Frontend - Testing and Coverage

Testing Approach:

- Vitest for unit tests with React Testing Library
- MSW (Mock Service Worker) for API mocking
- Component tests for all major UI components
- Form validation tests with various inputs
- Integration tests for user flows (login -> dashboard -> create transaction)
- Accessibility tests with jest-axe
- Coverage reports with Istanbul (target: >75%)

Task 26: Dockerization and Local Deployment

Docker Setup:

- Create Dockerfile for Spring Boot (multi-stage build)
- Create Dockerfile for React (Nginx serving static files)
- Update docker-compose.yml with all services
- Environment variable configuration for each service
- Health checks for all containers
- Persistent volumes for PostgreSQL data
- Docker network for service communication

Task 27: CI/CD Pipeline Setup

GitHub Actions Workflows:

- Backend: Lint, test, build, Docker image push
- Frontend: Lint, test, build, deploy to Vercel/Netlify
- Automated testing on pull requests
- Code coverage reporting with Codecov
- Automated dependency updates with Dependabot
- Security scanning with Snyk or GitHub Security

Task 28: Backend Deployment to Railway/Render

Deployment Steps:

- Create Railway/Render account and project
- Configure environment variables (database URL, JWT secret)
- Set up managed PostgreSQL database
- Deploy backend from Docker image or Git repository
- Configure custom domain (optional)
- Set up health check endpoints
- Configure automatic deployments from main branch
- Test API endpoints with Swagger UI

Task 29: Frontend Deployment to Vercel/Netlify

Deployment Steps:

- Connect Git repository to Vercel/Netlify
- Configure build command and output directory
- Set environment variables (API base URL)
- Configure rewrites for client-side routing

Expense Tracker - Implementation Plan

- Set up custom domain (optional)
- Enable automatic deployments from main branch
- Configure preview deployments for PRs
- Test deployed application end-to-end

Task 30: Documentation and Project Finalization

Documentation:

- README.md with project overview, setup instructions
- Architecture documentation with diagrams
- API documentation (Swagger + additional notes)
- Database schema documentation
- Deployment guide for each environment
- Contributing guidelines
- Security considerations and best practices
- Performance optimization notes
- Troubleshooting guide
- Demo video or screenshots

Implementation Timeline

Estimated timeline for full implementation:

Phase 1 - Foundation (Tasks 1-6): 2-3 weeks

- Project setup, database design, authentication, RBAC

Phase 2 - Backend APIs (Tasks 7-13): 3-4 weeks

- Core APIs, recurring detection, budgets, analytics, exports

Phase 3 - Backend Testing (Tasks 14-15): 1 week

- Documentation, comprehensive testing, coverage

Phase 4 - Frontend Core (Tasks 16-20): 2-3 weeks

- Setup, authentication UI, dashboard, transactions

Phase 5 - Frontend Features (Tasks 21-24): 2-3 weeks

- Categories, budgets, analytics, households

Phase 6 - Testing & Deployment (Tasks 25-30): 2 weeks

- Frontend testing, Docker, CI/CD, deployment, documentation

Total Estimated Time: 12-16 weeks (3-4 months) for complete implementation

This timeline assumes a single full-time developer. With a team or part-time work, adjust accordingly.

Conclusion

This implementation plan provides a comprehensive roadmap for building a production-ready, full-stack expense tracker application with advanced features including:

- Multi-user collaboration with role-based access control
- Automatic recurring transaction detection
- Advanced budget management with rollover
- Rule-based predictive analytics
- Comprehensive data visualization
- Professional export capabilities

The architecture emphasizes:

- Security: JWT authentication, role-based authorization, input validation
- Scalability: Stateless backend, efficient caching, database indexing
- Maintainability: Clean code structure, comprehensive testing, documentation
- Developer Experience: Modern tooling, hot reloading, type safety
- Production Readiness: Docker deployment, CI/CD pipelines, monitoring

The modular task breakdown allows for flexible implementation, enabling teams to prioritize features based on business requirements while maintaining a clear path to completion.

Key Success Factors

- Follow test-driven development: Write tests before implementation
- Maintain consistent code style: Use linters and formatters
- Document as you go: Don't defer documentation to the end
- Review security implications: Every feature should be security-reviewed
- Optimize performance: Profile and optimize database queries early
- User feedback: Test with real users during development
- Version control discipline: Meaningful commits, feature branches, PR reviews
- Monitoring and logging: Implement from day one, not as an afterthought

Potential Extensions

- Mobile applications: React Native or Flutter apps
- Receipt scanning: OCR integration for automatic transaction entry
- Bank integration: Connect to banking APIs for automatic imports
- Advanced ML predictions: Implement machine learning models for better forecasts
- Social features: Share achievements, spending challenges with friends
- Investment tracking: Expand to track investments and net worth
- Tax reporting: Generate tax-ready reports for specific jurisdictions
- Multi-currency support: Handle international transactions
- Notifications: Email/SMS/Push notifications for budget alerts
- API webhooks: Allow third-party integrations

Resources and References

Technical Documentation:

- Spring Boot: <https://spring.io/projects/spring-boot>
- React: <https://react.dev/>

Expense Tracker - Implementation Plan

- TanStack Query: <https://tanstack.com/query>
- shadcn/ui: <https://ui.shadcn.com/>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Flyway: <https://flywaydb.org/documentation/>
- Docker: <https://docs.docker.com/>

Best Practices:

- Spring Security: <https://spring.io/guides/topicals/spring-security-architecture>
- JWT: <https://jwt.io/introduction>
- REST API Design: <https://restfulapi.net/>
- React Patterns: <https://reactpatterns.com/>
- TypeScript: <https://www.typescriptlang.org/docs/handbook/>

Deployment Platforms:

- Vercel: <https://vercel.com/docs>
- Netlify: <https://docs.netlify.com/>
- Railway: <https://docs.railway.app/>
- Render: <https://render.com/docs>

End of Implementation Plan

Built with dedication for a robust expense tracking solution